

Day 13: Interrupts & ISR Safety

Platform: nRF52840 | **RTOS:** Zephyr RTOS

Learning Objectives

Zephyr IRQ API

Zephyr provides a high-level interrupt API. Most peripheral interrupts are configured using the GPIO, UART, or timer driver APIs, which internally configure the NVIC:

```
// GPIO interrupt (most common pattern)
gpio_pin_interrupt_configure_dt(&btn, GPIO_INT_EDGE_TO_ACTIVE);
gpio_init_callback(&cb, my_isr, BIT(btn.pin));
gpio_add_callback(btn.port, &cb);
```

For direct IRQ control (rare, advanced):

```
IRQ_CONNECT(DT_IRQ(DT_NODELABEL(uart0), irq), CONFIG_UART_NRF52_IRQ_PRIORITY, my_isr, NULL, 0);
irq_enable(DT_IRQ(DT_NODELABEL(uart0), irq));
```

ISR_DIRECT_DECLARE for Minimal Latency

ISR_DIRECT_DECLARE creates an ISR that bypasses Zephyr's normal interrupt wrapper — no context switching overhead. Use only for hard real-time peripherals where every microsecond counts:

```
ISR_DIRECT_DECLARE(my_fast_isr) {
    // Absolutely minimal work here
    ISR_DIRECT_PM(); // Notify PM system
    return 1;        // Request rescheduling
}
```

volatile in ISR Context

Variables shared between an ISR and main thread **must** be declared `volatile`. Without it, the compiler may cache the value in a register and never see the ISR's update:

```
static volatile uint32_t isr_event_count = 0;    // ISR increments, main reads
static volatile bool data_ready = false;        // ISR sets, main checks
```

irq_lock() / irq_unlock() — Critical Sections

To read/modify an ISR-shared variable atomically (prevent the ISR from running during the operation):

```
unsigned int key = irq_lock();    // Disable ALL interrupts
shared_var++;                    // Atomic operation
irq_unlock(key);                 // Re-enable interrupts
```

Keep critical sections **as short as possible** — every cycle spent with interrupts disabled adds latency to other interrupts.

k_sem_give from ISR — Safe Thread Communication

Never call `printk`, `k_mutex_lock`, `k_msleep`, or any blocking function from an ISR. Instead, use ISR-safe Zephyr primitives to communicate with threads:

```
// ISR:
k_sem_give(&data_ready_sem);    // Safe: non-blocking, ISR-safe

// Thread:
k_sem_take(&data_ready_sem, K_FOREVER); // Blocks until ISR gives
// Process the data here
```

GPIOTE on nRF52840

The nRF52840's **GPIOTE** (GPIO Tasks and Events) peripheral handles GPIO interrupts and tasks. It can:

- Generate an interrupt on pin edge (rising/falling/both)
- Generate a hardware task (e.g., toggle a pin) without CPU involvement
- Drive PWM via PPI (Programmable Peripheral Interconnect)

Zephyr's GPIO driver uses GPIOTE internally when you call `gpio_pin_interrupt_configure_dt()`.

Practice Tasks

1. Write an ISR that counts button presses and communicates the count to main via `K_MSGQ`
 2. Measure ISR latency: set a pin HIGH at ISR entry, LOW at exit — measure with oscilloscope or logic analyzer
 3. Nest two interrupts at different priorities — verify the higher priority preempts the lower
 4. Use `irq_lock()` / `irq_unlock()` to protect a 64-bit counter update on the 32-bit Cortex-M4
-

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day13
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)
- [nRF52840 Product Specification](#)
- [Nordic DevZone](#)
- Source: [src/day13/main.cpp](#)